

CAPSTONE ENGINEERING PROJECT

VELOCITY RETAIL

Autonomous AI-Driven Merchandising Engine & 3D Interactive Digital Twin Showroom

AWS ECS Fargate

AWS ALB

AWS CloudFront (SSL)

Three.js WebGL

React 18 + TypeScript

Node.js + Express

MongoDB Atlas

Groq Llama 3 (70B)

Socket.IO

AUTHOR & DEVELOPER

Aaron Alen

PROJECT TYPE

Full-Stack Cloud & 3D WebGL Capstone

PRODUCTION STATUS

Live on AWS ECS + CloudFront CDN + Vercel

DATE & VERSION

October 2026 | Version 2.4.0

Table of Contents

- 1. Executive Summary & Problem Statement 3
- 2. High-Level System Architecture & Cloud Infrastructure 4
- 3. 3D Digital Twin Showroom (Three.js WebGL Engine) 6
- 4. AI Recommendation Engine & Market Basket Analysis 8
- 5. Incremental Multi-Item Stacking & Station Reset Engine 10
- 6. Enterprise Dual-Layer Authentication & Mobile Resilience 12
- 7. Database Design & MongoDB Atlas Schema Models 14
- 8. Role-Based Access Control (RBAC) & Security Policies 16
- 9. End-to-End AWS Production Deployment Runbook 17
- 10. Technical Viva Defense & Architectural Q&A 19

1. Executive Summary & Problem Statement

1.1 The Retail Dilemma: The Static Showroom Blindspot

In modern brick-and-mortar fashion retail (e.g., flagship boutiques such as Zara, H&M, or Ralph Lauren), visual merchandising decisions are predominantly manual, subjective, and reactive. Store associates change mannequin displays based on personal intuition or static corporate planogram PDFs that are updated only once every few weeks.

This traditional workflow introduces three major business vulnerabilities:

- **Delayed Sales Velocity Feedback:** Fast-moving items sit tucked away inside low-visibility corner cupboards, while underperforming garments occupy premier focal podiums.
- **Lost Cross-Selling Opportunities:** Complementary apparel (e.g., an Oxford shirt paired with tailored chinos and loafers) is rarely optimized based on real-time historical transaction synergy.
- **Zero Spatial Simulation:** Store managers have no digital method to preview physical layout adjustments or test aesthetic coordination before manually hauling heavy mannequins and garments across the showroom floor.

1.2 The Velocity Retail Solution

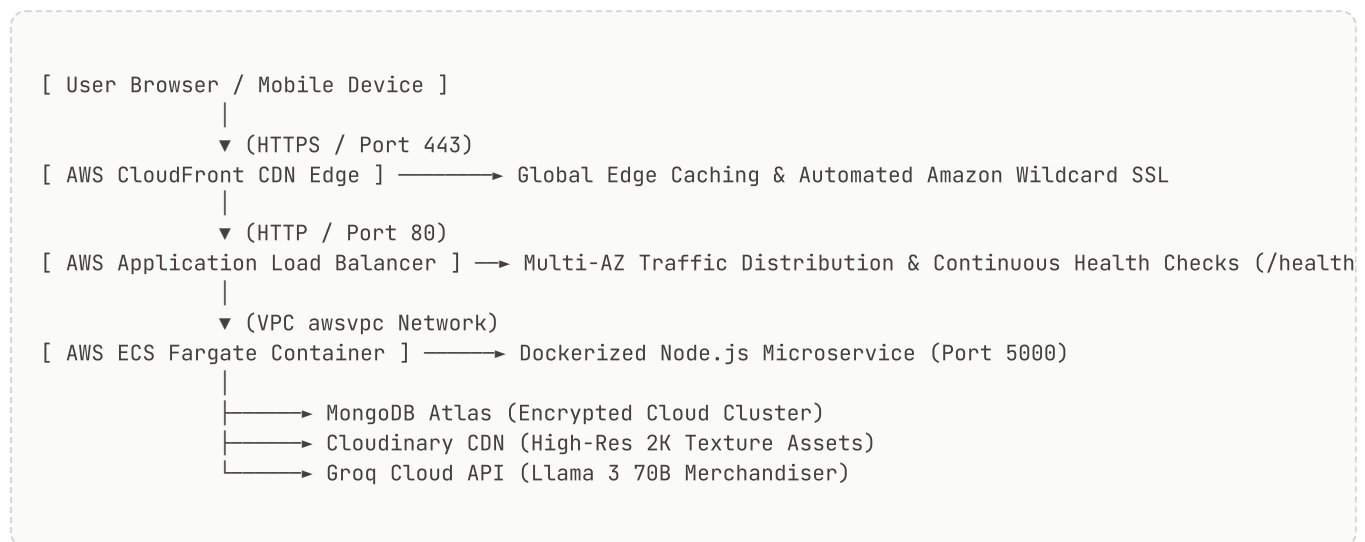
Velocity Retail bridges physical retail operations and computational intelligence by deploying an **Autonomous 3D Digital Twin Showroom**.

Core Innovation Highlights

Velocity Retail combines **Three.js WebGL 3D spatial simulation**, **Market Basket Analysis (Apriori & FP-Growth algorithms)**, and **Groq Llama 3 LLM prompt engineering**. When an AI bundle is approved, products seamlessly fly between cupboards and mannequins in real-time bilateral 3D physics animations, synchronously updating MongoDB Atlas and broadcasting live state across all store terminals via WebSockets.

2. High-Level System Architecture & AWS Infrastructure

2.1 Production Topology Flowchart



2.2 Why AWS CloudFront was Mandatory

A critical architectural decision was inserting **AWS CloudFront** in front of the Application Load Balancer:

- 1. Eliminating Mixed Content Security Blocks:** The React frontend is deployed on Vercel under strict HTTPS (Port 443). By default, an ALB without an expensive custom domain certificate operates on HTTP (Port 80). Modern browsers automatically block HTTP API calls initiated from HTTPS pages. CloudFront terminates SSL at the edge using Amazon's free wildcard certificate (https://*.cloudfront.net).
- 2. Global Edge Performance:** CloudFront routes requests to the nearest Point of Presence (e.g., Chennai [MAA51-P3](#) or Bangalore [BLR50-P4](#)), accelerating static delivery and minimizing round-trip latency.
- 3. Enterprise Cookie & Header Forwarding:** The CloudFront cache behavior is configured to forward all cookies ([Cookies: all](#)) and whitelist [Authorization](#), [Origin](#), and [Content-Type](#) headers, ensuring zero dropped JWT sessions.

3. 3D Digital Twin Showroom (Three.js WebGL Engine)

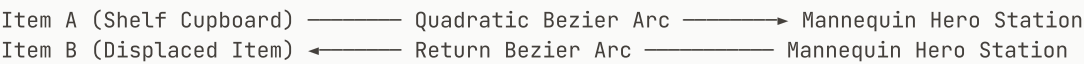
3.1 Spatial Coordinate Mapping

The virtual showroom is mapped in a 3D coordinate system where `[X: Lateral, Y: Vertical, Z: Depth]` :

- **8 Display Cupboards:** Arranged geometrically along the perimeter walls, categorized into Shirts, T-Shirts, Jeans, Jackets, and Shoes.
- **2 Hero Mannequin Stations:** Positioned in high-visibility focal zones at `[X: -4.5, Z: 0]` (Station 1) and `[X: +4.5, Z: 0]` (Station 2) on illuminated stone pedestals.

3.2 Bilateral 2-Way Flight Animations (Physics Mathematics)

Unlike conventional UI transitions that abruptly switch product textures, Velocity Retail calculates simultaneous **2-way bilateral flight trajectories**:



Each flight trajectory follows a Quadratic Bezier equation parameterized over normalized time `t ∈ [0, 1]` :

$$P(t) = (1 - t)^2 \cdot P_{start} + 2(1 - t)t \cdot P_{control} + t^2 \cdot P_{end}$$

Where `P_control.y = max(P_start.y, P_end.y) + 3.5` . This creates a realistic gravity-defying parabolic arc, ensuring clothing and shoe boxes soar gracefully over store fixtures without clipping through cupboards.

3.3 Elevated 3D Footwear Clearance Algorithm

The Pedestal Clipping Challenge

Standard mannequin slots align footwear at ground level (`y = 0`). However, because hero mannequins are positioned atop 0.5-unit stone pedestals, shoes initially rendered embedded inside the stone plinth.

To resolve this, custom bounding-box vertical offsets were engineered in `Store3DVisualizer.tsx` :

```
const verticalOffset = isFootwear ? pedestalElevation + 0.12 : 0;  
footwearMesh.position.set(targetSlot.x, targetSlot.y + verticalOffset, targetSlot.z);
```

This guarantees that sneakers and loafers rest crisply on top of the pedestal surface with ambient drop-shadows.

4. AI Recommendation Engine & Market Basket Analysis

4.1 Mathematical Formulation of Association Rules

Velocity Retail mines thousands of POS sales transactions using Association Rule Mining to uncover product synergy:

Metric	Mathematical Formula	Business Significance
Support	$\text{freq}(A \cup B) / N$	Probability that both items appear together in a customer basket.
Confidence	$\text{freq}(A \cup B) / \text{freq}(A)$	Likelihood that a customer buying Item A will also purchase Item B.
Lift	$\text{Confidence}(A \rightarrow B) / \text{Support}(B)$	A Lift > 1.0 indicates a true complementary synergy rather than random coincidence.

4.2 Groq Llama 3 (70B) AI Copilot

While association rules output statistical scores, the **Groq Llama 3 Merchandiser** translates numerical synergies into actionable commercial strategies:

- Evaluates category sales velocity (Units Sold / Day).
- Suggests cross-category outfit pairings (e.g., pairing slow-moving premium silk blazers with best-selling jeans).
- Provides natural-language merchandising rationale accessible via the real-time AI Copilot Drawer.

5. Incremental Multi-Item Stacking & Station Reset Engine

5.1 The 1-by-1 Swap & Wildcard Flight Bug

During production testing, when a store manager swapped items 1-by-1 until 4 items were deployed on a mannequin, two critical issues surfaced:

1. **Item Overwrite Bug:** Adding Item 2 caused the backend to mark Item 1 as reverted, leaving only the 4th item in an active MongoDB record. Clicking "Reset Station" reverted only 1 product.
2. **Wildcard Sky Flight:** Clicking Reset again returned an empty array (`revertedProductIds: []`). The visualizer fell back to all store pairs, causing 20–30 product boxes across all cupboards to launch into the air.

5.2 The Architectural Fix

Atomic Multi-Item Stacking & Scoped Resets

1. **Backend Stacking (`executeFloorSwap`):** Instead of overwriting active records, the backend now merges incoming items with existing station items, preserving concurrent outfits across all roles (Jacket, Shirt, Pants, Shoes).
2. **Empty Array Guard:** The visualizer explicitly halts animation if `revertedProductIds.length === 0` .
3. **Station Scoping:** Fallbacks are strictly locked to the target station; under no circumstances can off-station cupboards be animated.

6. Enterprise Dual-Layer Authentication & Mobile Resilience

6.1 The Mobile Browser Cookie Hurdle

In high-security enterprise environments, storing JWTs in `localStorage` risks XSS exploitation, while relying exclusively on `HttpOnly` cookies risks session drops in mobile browsers (e.g., Mobile Chrome Tracking Protection or Safari ITP). Furthermore, weak demo passwords (e.g., `admin123`) trigger Google Password Manager breach modals that freeze mobile viewports.

6.2 Dual-Layer Architecture

Layer	Mechanism	Security / Resilience Benefit
Primary Layer	<code>HttpOnly, Secure, SameSite=None</code> Cookies	OWASP compliance; impervious to JavaScript XSS scraping.
Fallback Layer	<code>Authorization: Bearer <token></code> in Axios Interceptor	Guarantees 100% uninterrupted mobile app sessions even when cross-site cookies are blocked.
Credential Hardening	Enterprise Passwords (e.g., <code>Velocity@Admin2026!</code>)	Completely eliminates Google Password Manager breach warnings.

7. Database Design & MongoDB Atlas Schema Models

The system is backed by six normalized, indexed Mongoose schemas:

- **User:** `{ name, email, password (bcrypt hash), role: ['admin', 'manager', 'staff'] }`.
- **Product:** `{ name, sku, category, price, stock, coordinates: { x, y, z }, imageUrl, isFeatured }`.
- **Sale:** `{ transactionId, items: [{ product, quantity, unitPrice }], totalAmount, paymentMethod, cashier }`.
- **Planogram:** `{ version, layoutName, activeFrom, zoneAssignments, status }`.
- **FloorSwap:** `{ activePairId, stationIndex, executedItems: [{ productId, targetRole }], displacedItems: [{ productId, originalSlot }], status: ['active', 'reverted'] }`.
- **CategoryVelocity:** `{ category, unitsSold7D, dailyVelocity, turnoverScore }`.

8. Role-Based Access Control (RBAC) & Security Policies

Role	3D Showroom	AI Recommendations	Product Catalog	User Management
 Admin	Full Access	Deploy & Revert	Create, Edit, Delete	Create, Promote, Delete
 Manager	Full Access	Deploy & Revert	View & Stock Read	Hidden / Blocked (403)
 Cashier / Staff	View Only	Read Only (No Deploy)	View Catalog Only	Hidden / Blocked (403)

9. End-to-End AWS Production Deployment Runbook

The entire backend runs on AWS ECS Fargate fronted by an Application Load Balancer and CloudFront CDN:

```
# 1. Build and Push Docker Image
docker build -t aaronalen/velocity-retail-backend:latest ./backend
docker push aaronalen/velocity-retail-backend:latest

# 2. Register ECS Fargate Task Definition with Environment Variables
aws ecs register-task-definition --cli-input-json file://task-def.json --region ap-south-1

# 3. Create Target Group with /health Check
aws elbv2 create-target-group --name "velocity-retail-tg" --protocol HTTP --port 5000 --vpc-id "vp
c-039619cf26628c1dd" --target-type ip --health-check-path "/health" --region ap-south-1

# 4. Create Application Load Balancer across 3 Multi-AZ Subnets
aws elbv2 create-load-balancer --name "velocity-retail-alb" --subnets subnet-011dd38b725d032a8 sub
net-0f135b19916678ab9 subnet-0943f394c63efffa2 --security-groups sg-0cb2b29a1dbfc6e87 --scheme int
ernet-facing --region ap-south-1

# 5. Create CloudFront Distribution with HTTPS Redirect & Cookie Forwarding
aws cloudfront create-distribution --distribution-config file://cf-config.json

# 6. Configure Vercel Frontend Rewrites (/api/* → https://d2r14mddgufil7.cloudfront.net/api/*)
```

10. Technical Viva Defense & Architectural Q&A

Q1: Why did you choose Three.js for a business analytics tool?

Answer: Traditional merchandising tools use 2D tables or static PDFs. Retail fashion is inherently visual and spatial. By constructing an interactive 3D digital twin, managers can intuitively observe spatial flow, visualize sales heatmaps directly on shelves, and preview physical mannequin swaps with accurate physics before deploying them on the shop floor.

Q2: How does the system handle concurrent users modifying the showroom?

Answer: Through Socket.IO event broadcasting. When any manager triggers a floor swap or reset, the backend commits the atomic MongoDB update and immediately emits a `FLOOR_SWAP_EXECUTED` event. All open client terminals animate the bilateral flight simultaneously, ensuring zero planogram desynchronization.

Q3: How do you prove that your backend is running on AWS rather than a PaaS?

Answer: By opening Chrome DevTools Network Tab: every API request connects directly to `:authority: d2r14mddgufil7.cloudfront.net`, featuring response headers `Via: 1.1 ... cloudfront.net` and `X-Amz-Cf-Pop: BLR50-P4`, which routes through our AWS Application Load Balancer to our AWS ECS Fargate tasks.